

Lost & Found Intelligence: Complete Implementation Guide

Prepared for Haseena Syed, October 8, 2026

1. What you are building

A web platform where people report lost or found objects using text, a photo, a place and a time. A multi-factor matching engine suggests likely matches, and an item is released only after private ownership verification. This guide takes you from an empty folder to a deployed, working site, with code for every core piece.

Must-have (build first): accounts and roles; lost and found reporting; text and image embeddings; match suggestions with a score breakdown; hidden-detail ownership verification; handover code; moderator queue.

Stretch: learned ranker (LightGBM); analytics dashboard; email notifications; Hinglish tuning.

Design rules:

- The system suggests matches. It never declares ownership.
- Hidden details are encrypted and never sent to a claimant or shown publicly.
- Every sensitive action is written to an audit log.

The synthetic dataset I generated earlier (lost_found_dataset.zip) is used for seeding the demo and for evaluating the matcher.

2. Architecture and stack

Layer	Choice	Why
Frontend	React 18, Vite, Tailwind CSS	Fast to build, matches your project note
Backend	FastAPI, SQLAlchemy 2, Pydantic	Typed, automatic API docs at /docs
Database	PostgreSQL 16 with pgvector	Relational data and vector search in one place

Layer	Choice	Why
Embeddings	sentence-transformers: clip-ViT-B-32 (images and text in one 512-d space) and paraphrase-multilingual-MiniLM-L12-v2 (descriptions, 384-d, copes with Hinglish)	Cross-modal search plus strong text matching
Images	Pillow, OpenCV (face blur), optional Tesseract OCR	Privacy and extra match signals
Auth	JWT (PyJWT), argon2 password hashes	Standard and simple
Encryption	cryptography Fernet	Encrypts hidden details and claim answers
Deployment	Docker Compose, Caddy for HTTPS	One command to run anywhere

Request flow

1. User submits a report (form plus optional photo).
2. API validates input, strips photo metadata, blurs faces, stores a public and a private copy.
3. Embeddings are computed and saved on the report.
4. A background task retrieves candidate reports of the opposite type with pgvector, then re-ranks them with the multi-factor score.
5. Match rows are created and both users are notified.
6. The owner starts a claim and receives three private questions.
7. Answers are scored: 3 correct approves, 2 goes to a moderator, fewer rejects.
8. On approval the claimant gets a one-time handover code. The finder enters it at handover and both reports close.

3. Repository layout

```
lost-found/
  docker-compose.yml
  .env
  data/                                # unzip lost_found_dataset.zip here (CSVs +
gen_dataset.py)                        # stretch: LightGBM ranker
  scripts/
    export_questions.py                # builds backend/app/data/questions.json
    seed.py                            # loads places, demo users and sample reports
    train_ranker.py
```

```

    evaluate.py                # Recall@k and MRR on the test split
backend/
  Dockerfile
  requirements.txt
app/
  main.py
  db.py
  models.py
  schemas.py
  core/{config.py, security.py}
  services/{embeddings.py, imaging.py, matching.py, verification.py,
notify.py}
  api/{auth.py, reports.py, search.py, claims.py, admin.py}
  data/questions.json
  tests/
frontend/
  Dockerfile
  package.json
  src/{main.jsx, App.jsx, api.js, pages/, components/}

```

Create the empty folders and files first so the imports below resolve. Add an empty `__init__.py` to `app`, `app/core`, `app/services` and `app/api`.

4. Environment setup

Prerequisites: Docker Desktop, Git, Python 3.11 or newer, Node 20 or newer.

Generate secrets

```

python -c 'from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())'
python -c 'import secrets; print(secrets.token_urlsafe(48))'

```

`.env` (never commit this file)

```

DATABASE_URL=postgresql+psycopg://lf:lfpass@db:5432/lostfound
JWT_SECRET=paste-the-long-random-string
FERNET_KEY=paste-the-fernet-key
CORS_ORIGINS=http://localhost:5173
UPLOAD_DIR=/data/uploads

```

`docker-compose.yml`

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: lf
      POSTGRES_PASSWORD: lfpass
      POSTGRES_DB: lostfound
    volumes:
      - dbdata:/var/lib/postgresql/data
    ports:
      - '5432:5432'
  backend:
    build: ./backend
    env_file: .env
    depends_on:
      - db
    volumes:
      - uploads:/data/uploads
      - ./backend:/code
      - ./data:/data/seed
      - ./scripts:/code/scripts
    ports:
      - '8000:8000'
  frontend:
    build: ./frontend
    environment:
      VITE_API_URL: http://localhost:8000
    volumes:
      - ./frontend:/app
      - /app/node_modules
    ports:
      - '5173:5173'
volumes:
  dbdata:
  uploads:
```

backend/Dockerfile

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends tesseract-ocr libgl1 libglu1-mesa-dev && rm -rf /var/lib/apt/lists/*
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

frontend/Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD npm run dev -- --host 0.0.0.0
```

backend/requirements.txt (starting versions; upgrade once everything works)

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
sqlalchemy==2.0.35
psycopg[binary]==3.2.3
pgvector==0.3.5
pydantic-settings==2.5.2
pyjwt==2.9.0
argon2-cffi==23.1.0
cryptography==43.0.1
python-multipart==0.0.12
pillow==10.4.0
opencv-python-headless==4.10.0.84
sentence-transformers==3.1.1
pytesseract==0.3.13
numpy==1.26.4
pandas==2.2.2
lightgbm==4.5.0
scikit-learn==1.5.2
pytest==8.3.3
httpx==0.27.2
```

Bring everything up with `docker compose up --build`. The API docs appear at <http://localhost:8000/docs> once the backend code below exists.

5. Backend foundation

5.1 backend/app/core/config.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL: str = 'postgresql+psycopg://lf:lfpass@db:5432/lostfound'
    JWT_SECRET: str = 'change-me'
    JWT_MINUTES: int = 120
    FERNET_KEY: str = ''
    UPLOAD_DIR: str = '/data/uploads'
    MAX_UPLOAD_MB: int = 5
    CORS_ORIGINS: str = 'http://localhost:5173'
    MATCH_MIN_SCORE: float = 0.45
    MAX_CLAIM_ATTEMPTS: int = 2

    class Config:
        env_file = '.env'

settings = Settings()
```

5.2 backend/app/db.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from .core.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False,
                             expire_on_commit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

5.3 backend/app/models.py

```
import datetime as dt

from pgvector.sqlalchemy import Vector
from sqlalchemy import (JSON, Boolean, DateTime, Float, ForeignKey, Index,
                        Integer,
                        LargeBinary, String, Text, UniqueConstraint)
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column,
relationship

IMG_DIM, TXT_DIM = 512, 384

def now():
    return dt.datetime.now(dt.timezone.utc).replace(tzinfo=None)

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(120))
    password_hash: Mapped[str] = mapped_column(String(255))
    role: Mapped[str] = mapped_column(String(20), default='user') # user |
moderator
    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)

class Place(Base):
    __tablename__ = 'places'
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), unique=True)
    latitude: Mapped[float] = mapped_column(Float)
    longitude: Mapped[float] = mapped_column(Float)

class Report(Base):
    __tablename__ = 'reports'
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey('users.id'), index=True)
    type: Mapped[str] = mapped_column(String(10), index=True) # lost | found
```

```

category: Mapped[str] = mapped_column(String(40), index=True)
group: Mapped[str] = mapped_column(String(40), index=True)
brand: Mapped[str] = mapped_column(String(80), default='')
color: Mapped[str] = mapped_column(String(40), default='')
description: Mapped[str] = mapped_column(Text)
place_id: Mapped[int] = mapped_column(ForeignKey('places.id'))
event_time: Mapped[dt.datetime] = mapped_column(DateTime, index=True)
time_window_hours: Mapped[float] = mapped_column(Float, default=1.0)
status: Mapped[str] = mapped_column(String(20), default='open',
index=True) # open | claimed | closed
ocr_text: Mapped[str] = mapped_column(Text, default='')
img_emb = mapped_column(Vector(IMG_DIM), nullable=True) # CLIP image
clip_txt_emb = mapped_column(Vector(IMG_DIM), nullable=True) # CLIP text
(cross-modal)
txt_emb = mapped_column(Vector(TXT_DIM), nullable=True) #
multilingual text
created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)
closed_at: Mapped[dt.datetime | None] = mapped_column(DateTime,
nullable=True)

place = relationship('Place')
images = relationship('ReportImage', back_populates='report',
cascade='all, delete-orphan')
hidden = relationship('HiddenDetail', uselist=False, cascade='all,
delete-orphan')

__table_args__ = (
    Index('ix_reports_img', 'img_emb', postgresql_using='hnsw',
postgresql_ops={'img_emb': 'vector_cosine_ops'}),
    Index('ix_reports_txt', 'txt_emb', postgresql_using='hnsw',
postgresql_ops={'txt_emb': 'vector_cosine_ops'}),
)

class ReportImage(Base):
    __tablename__ = 'report_images'
    id: Mapped[int] = mapped_column(primary_key=True)
    report_id: Mapped[int] = mapped_column(ForeignKey('reports.id'),
index=True)
    private_path: Mapped[str] = mapped_column(String(300)) # original, never
served publicly
    public_path: Mapped[str] = mapped_column(String(300)) # faces blurred,
metadata stripped
    report = relationship('Report', back_populates='images')

```



```

class HiddenDetail(Base):
    __tablename__ = 'hidden_details'
    report_id: Mapped[int] = mapped_column(ForeignKey('reports.id'),
primary_key=True)
    payload: Mapped[bytes] = mapped_column(LargeBinary) # Fernet-encrypted
JSON {key: value}

class Match(Base):
    __tablename__ = 'matches'
    id: Mapped[int] = mapped_column(primary_key=True)
    lost_id: Mapped[int] = mapped_column(ForeignKey('reports.id'),
index=True)
    found_id: Mapped[int] = mapped_column(ForeignKey('reports.id'),
index=True)
    score: Mapped[float] = mapped_column(Float)
    components: Mapped[dict] = mapped_column(JSON, default=dict)
    confidence: Mapped[str] = mapped_column(String(10)) # high | medium |
low
    status: Mapped[str] = mapped_column(String(20), default='suggested') #
suggested | dismissed | claimed | closed
    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)
    lost = relationship('Report', foreign_keys=[lost_id])
    found = relationship('Report', foreign_keys=[found_id])
    __table_args__ = (UniqueConstraint('lost_id', 'found_id'),)

class Claim(Base):
    __tablename__ = 'claims'
    id: Mapped[int] = mapped_column(primary_key=True)
    match_id: Mapped[int] = mapped_column(ForeignKey('matches.id'),
index=True)
    claimant_id: Mapped[int] = mapped_column(ForeignKey('users.id'),
index=True)
    answers: Mapped[bytes | None] = mapped_column(LargeBinary, nullable=True)
# encrypted
    n_correct: Mapped[int] = mapped_column(Integer, default=0)
    attempts: Mapped[int] = mapped_column(Integer, default=0)
    decision: Mapped[str] = mapped_column(String(20), default='pending') #
pending | approved | manual_review | rejected
    handover_hash: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    decided_by: Mapped[int | None] = mapped_column(ForeignKey('users.id'),
nullable=True)
    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)
    match = relationship('Match')

```

```

class Notification(Base):
    __tablename__ = 'notifications'
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey('users.id'), index=True)
    text: Mapped[str] = mapped_column(String(300))
    link: Mapped[str] = mapped_column(String(200), default='')
    read: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)

class AuditLog(Base):
    __tablename__ = 'audit_log'
    id: Mapped[int] = mapped_column(primary_key=True)
    actor_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    action: Mapped[str] = mapped_column(String(60))
    entity: Mapped[str] = mapped_column(String(40))
    entity_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    detail: Mapped[dict] = mapped_column(JSON, default=dict)
    ts: Mapped[dt.datetime] = mapped_column(DateTime, default=now)

class AbuseReport(Base):
    __tablename__ = 'abuse_reports'
    id: Mapped[int] = mapped_column(primary_key=True)
    reporter_id: Mapped[int] = mapped_column(ForeignKey('users.id'))
    report_id: Mapped[int] = mapped_column(ForeignKey('reports.id'))
    reason: Mapped[str] = mapped_column(String(300))
    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=now)

```

Tables are created automatically at startup (section 8.6). When the schema starts changing often, add Alembic migrations.

5.4 backend/app/core/security.py

```

import datetime as dt
import json

import jwt
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
from cryptography.fernet import Fernet
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

```

```

from ..db import get_db
from ..models import User
from .config import settings

ph = PasswordHasher()
oauth2 = OAuth2PasswordBearer(tokenUrl='/auth/login')

def hash_password(p: str) -> str:
    return ph.hash(p)

def verify_password(hashed: str, p: str) -> bool:
    try:
        return ph.verify(hashed, p)
    except VerifyMismatchError:
        return False

def make_token(user_id: int, role: str) -> str:
    exp = dt.datetime.now(dt.timezone.utc) +
dt.timedelta(minutes=settings.JWT_MINUTES)
    return jwt.encode({'sub': str(user_id), 'role': role, 'exp': exp},
settings.JWT_SECRET, algorithm='HS256')

def _fernet() -> Fernet:
    return Fernet(settings.FERNET_KEY.encode())

def encrypt_json(obj) -> bytes:
    return _fernet().encrypt(json.dumps(obj).encode())

def decrypt_json(blob: bytes):
    return json.loads(_fernet().decrypt(blob))

def current_user(token: str = Depends(oauth2), db: Session = Depends(get_db))
-> User:
    try:
        data = jwt.decode(token, settings.JWT_SECRET, algorithms=['HS256'])
    except jwt.PyJWTError:
        raise HTTPException(401, 'Invalid or expired token')
    user = db.get(User, int(data['sub']))

```

```

    if not user:
        raise HTTPException(401, 'User not found')
    return user

def require_moderator(user: User = Depends(current_user)) -> User:
    if user.role != 'moderator':
        raise HTTPException(403, 'Moderator access required')
    return user

```

6. Question templates for ownership verification

The dataset generator already holds the private-detail questions for all 14 categories. Export them once so the backend asks the same questions the dataset was built around.

scripts/export_questions.py (run from the project root: `python scripts/export_questions.py`)

```

import json
import sys

sys.path.insert(0, 'data') # the folder containing gen_dataset.py
from gen_dataset import CATS

out = {
    cat: {
        'group': spec['group'],
        'questions': {k: {'q': v[0], 'options': v[1]} for k, v in
spec['hidden'].items()},
    }
    for cat, spec in CATS.items()
}
json.dump(out, open('backend/app/data/questions.json', 'w'), indent=2)
print('categories:', len(out))

```

The `options` list is used in two places: to populate the finder's form (a dropdown or free text) and, in the verifier, to find which option a claimant free-text answer is closest to. Options are never sent to a claimant.

7. Service layer

7.1 backend/app/services/embeddings.py

```
from functools import lru_cache

import numpy as np
from PIL import Image
from sentence_transformers import SentenceTransformer

@lru_cache(maxsize=1)
def clip_model():
    return SentenceTransformer('clip-ViT-B-32')

@lru_cache(maxsize=1)
def text_model():
    return SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

def _norm(v):
    v = np.asarray(v, dtype='float32')
    return (v / (np.linalg.norm(v) + 1e-9)).tolist()

def embed_text(text):
    return _norm(text_model().encode(text))

def embed_clip_text(text):
    return _norm(clip_model().encode(text)) # CLIP truncates long text,
which is fine here

def embed_image(path_or_file):
    return
_norm(clip_model().encode(Image.open(path_or_file).convert('RGB')))
```

The first call downloads the models (a few hundred MB). Mount a volume for the Hugging Face cache, or pre-download in the Dockerfile, so rebuilds stay fast.

7.2 backend/app/services/imaging.py

```

import io
import os
import uuid

import cv2
import numpy as np
from fastapi import HTTPException
from PIL import Image

from ..core.config import settings

ALLOWED = {'image/jpeg', 'image/png', 'image/webp'}
_face = cv2.CascadeClassifier(cv2.data.haarcascades +
                              'haarcascade_frontalface_default.xml')

def save_image(raw: bytes, content_type: str):
    if content_type not in ALLOWED:
        raise HTTPException(400, 'Only JPEG, PNG or WebP images are allowed')
    if len(raw) > settings.MAX_UPLOAD_MB * 1024 * 1024:
        raise HTTPException(400, 'Image too large')
    try:
        img = Image.open(io.BytesIO(raw)).convert('RGB')
    except Exception:
        raise HTTPException(400, 'Invalid image')
    img.thumbnail((1280, 1280))
    os.makedirs(settings.UPLOAD_DIR + '/private', exist_ok=True)
    os.makedirs(settings.UPLOAD_DIR + '/public', exist_ok=True)
    name = uuid.uuid4().hex + '.jpg'
    private = settings.UPLOAD_DIR + '/private/' + name
    img.save(private, 'JPEG', quality=88) # re-encoding drops EXIF and GPS
    metadata
    arr = cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)
    gray = cv2.cvtColor(arr, cv2.COLOR_BGR2GRAY)
    for (x, y, w, h) in _face.detectMultiScale(gray, 1.1, 5):
        arr[y:y + h, x:x + w] = cv2.GaussianBlur(arr[y:y + h, x:x + w], (51,
51), 0)
    public = settings.UPLOAD_DIR + '/public/' + name
    cv2.imwrite(public, arr)
    return private, public

def ocr_text(path):
    try:

```

```

import pytesseract
return pytesseract.image_to_string(Image.open(path)).strip()[:500]
except Exception:
return ''

```

Embeddings and OCR use the private original. Only the blurred copy is served. A later improvement is to also blur ID cards and phone screens, for example with a small YOLO model trained on those classes.

7.3 backend/app/services/notify.py

```

from ..models import AuditLog, Notification

def notify(db, user_id, text, link=''):
    db.add(Notification(user_id=user_id, text=text, link=link))

def audit(db, actor_id, action, entity, entity_id=None, **detail):
    db.add(AuditLog(actor_id=actor_id, action=action, entity=entity,
entity_id=entity_id, detail=detail))

```

7.4 backend/app/services/matching.py

This is the technical heart of the project. Version 1 is a transparent weighted score. Component scores are stored with every match so the UI can show why a match was suggested.

```

import math
import re

import numpy as np
from sqlalchemy import select
from sqlalchemy.orm import Session

from ..core.config import settings
from ..models import Match, Report

WEIGHTS = {'text': 0.30, 'image': 0.25, 'color': 0.08, 'brand': 0.07, 'geo':
0.15, 'time': 0.10, 'ocr': 0.05}
CROSS_MODAL_SCALE = 3.0 # CLIP text-to-image cosines are small; calibrate
this on the dataset
COLOR_FAMILY = {
    'black': 'dark', 'grey': 'dark', 'navy blue': 'blue', 'blue': 'blue',
    'red': 'red', 'maroon': 'red',

```

```

    'green': 'green', 'white': 'light', 'silver': 'light', 'brown': 'brown',
    'pink': 'red', 'orange': 'orange',
}

def haversine_m(lat1, lon1, lat2, lon2):
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dphi, dl = p2 - p1, math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl /
2) ** 2
    return 2 * 6371000.0 * math.asin(math.sqrt(a))

def cosine(a, b):
    if a is None or b is None:
        return None
    return float(np.dot(np.asarray(a), np.asarray(b))) # vectors are L2-
normalised

def color_score(a, b):
    if not a or not b:
        return None
    if a == b:
        return 1.0
    return 0.5 if a in COLOR_FAMILY and COLOR_FAMILY.get(a) ==
COLOR_FAMILY.get(b) else 0.0

def brand_score(a, b):
    if not a or not b:
        return 0.5 # unknown is neutral
    return 1.0 if a.lower() == b.lower() else 0.0

def tokens(s):
    return set(re.findall('[a-z0-9]{3,}', (s or '').lower()))

def ocr_score(a, b):
    ta, tb = tokens(a), tokens(b)
    if not ta or not tb:
        return None
    return len(ta & tb) / len(ta | tb)

```



```

def pair_components(lost, found):
    d = haversine_m(lost.place.latitude, lost.place.longitude,
found.place.latitude, found.place.longitude)
    hours = (found.event_time - lost.event_time).total_seconds() / 3600
    earliest = -(lost.time_window_hours / 2 + 2) # tolerate imprecise lost
times
    time_s = 0.0 if hours < earliest else math.exp(-max(hours, 0) / 30)
    comp = {
        'text': cosine(lost.txt_emb, found.txt_emb),
        'image': cosine(lost.img_emb, found.img_emb),
        'color': color_score(lost.color, found.color),
        'brand': brand_score(lost.brand, found.brand),
        'geo': math.exp(-d / 250),
        'time': time_s,
        'ocr': ocr_score(lost.ocr_text, found.ocr_text),
    }
    if comp['image'] is None: # one side has a photo, the other only words
        if lost.img_emb is None and found.img_emb is not None:
            c = cosine(lost.clip_txt_emb, found.img_emb)
        elif found.img_emb is None and lost.img_emb is not None:
            c = cosine(found.clip_txt_emb, lost.img_emb)
        else:
            c = None
        comp['image'] = None if c is None else c * CROSS_MODAL_SCALE
    return comp

```

```

def weighted_score(comp):
    num = den = 0.0
    for k, w in WEIGHTS.items():
        v = comp.get(k)
        if v is None:
            continue # missing signals are skipped and the rest re-
normalised
        num += w * max(0.0, min(1.0, v))
        den += w
    return num / den if den else 0.0

```

RANKER = None # replaced by a trained model in section 11

```

def score_pair(comp):
    if RANKER is not None:
        return float(RANKER(comp))
    return weighted_score(comp)

```

```

def confidence(score):
    return 'high' if score >= 0.75 else 'medium' if score >= 0.55 else 'low'

def candidates(db: Session, rep: Report, k: int = 50):
    opposite = 'found' if rep.type == 'lost' else 'lost'
    q = select(Report).where(Report.type == opposite, Report.status ==
'open', Report.user_id != rep.user_id)
    if rep.group != 'other':
        q = q.where(Report.group.in_([rep.group, 'other']))
    if rep.txt_emb is not None:
        q = q.order_by(Report.txt_emb.cosine_distance(rep.txt_emb))
    return db.scalars(q.limit(k)).all()

def run_matching(db: Session, report_id: int):
    rep = db.get(Report, report_id)
    if not rep or rep.status != 'open':
        return []
    scored = []
    for other in candidates(db, rep):
        lost, found = (rep, other) if rep.type == 'lost' else (other, rep)
        comp = pair_components(lost, found)
        s = score_pair(comp)
        if s >= settings.MATCH_MIN_SCORE:
            scored.append((lost, found, comp, s))
    scored.sort(key=lambda t: -t[3])
    created = []
    for lost, found, comp, s in scored[:5]:
        if db.scalar(select(Match).where(Match.lost_id == lost.id,
Match.found_id == found.id)):
            continue
        m = Match(lost_id=lost.id, found_id=found.id, score=round(s, 4),
confidence=confidence(s),
                    components={k: (None if v is None else round(v, 4)) for k,
v in comp.items()})
        db.add(m)
        created.append(m)
    db.commit()
    return created

```

The candidate filter uses the category group, an open status, the opposite report type and a different user. Time and place are scored rather than used as hard filters, so a wrong-place report can still surface with a lower score.

7.5 backend/app/services/verification.py

```

import json
import os
import secrets

import numpy as np

from .embeddings import text_model

_path = os.path.join(os.path.dirname(__file__), '..', 'data',
'questions.json')
QUESTIONS = json.load(open(_path))

def question_text(category, key):
    return QUESTIONS[category]['questions'][key]['q']

def norm(s):
    return ' '.join(str(s).lower().strip().split())

def is_correct(category, key, truth, answer):
    a, t = norm(answer), norm(truth)
    if not a:
        return False
    if a == t or (' ' + t + ' ') in (' ' + a + ' '):
        return True
    opts = QUESTIONS[category]['questions'][key]['options']
    vecs = text_model().encode([answer] + opts, normalize_embeddings=True)
    sims = vecs[1:] @ vecs[0]
    best = int(np.argmax(sims))
    return norm(opts[best]) == t and float(sims[best]) >= 0.45 # closest
known option must be the true one

def score_answers(category, hidden, answers):
    return {k: is_correct(category, k, truth, answers.get(k, '')) for k,
truth in hidden.items()}

def decision_from(n_correct, n_questions):
    if n_correct == n_questions:
        return 'approved'
    if n_questions >= 3 and n_correct == n_questions - 1:

```

```

        return 'manual_review'
    return 'rejected'

```

```

def new_handover_code():
    return ''.join(secrets.choice('0123456789') for _ in range(6))

```

The decision rule is the same one used to label the synthetic dataset, so you can replay `verification_claims.csv` through `is_correct` and report agreement with the labels and the false-approval rate. Known limitation: the substring check can accept an answer that negates the truth (for example not red). Mention it in your threats-to-validity section.

8. API layer

Add an empty file `backend/app/api/misc.py` to the layout. Routers return plain dictionaries through the helper below, which keeps public and private fields clearly separated.

8.1 backend/app/schemas.py

```

import os

def report_out(r, owner=False):
    d = {
        'id': r.id, 'type': r.type, 'category': r.category, 'brand': r.brand,
        'color': r.color,
        'description': r.description, 'place': r.place.name, 'event_time':
        r.event_time.isoformat(),
        'status': r.status,
        'images': ['/files/public/' + os.path.basename(i.public_path) for i
        in r.images],
    }
    if owner:
        d['time_window_hours'] = r.time_window_hours
    return d

```

8.2 backend/app/api/auth.py

```

from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordRequestForm
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.orm import Session

```

```

from ..core.security import hash_password, make_token, verify_password
from ..db import get_db
from ..models import User

router = APIRouter(prefix='/auth', tags=['auth'])

class RegisterIn(BaseModel):
    email: str
    name: str
    password: str

@router.post('/register')
def register(body: RegisterIn, db: Session = Depends(get_db)):
    if len(body.password) < 8:
        raise HTTPException(400, 'Password must be at least 8 characters')
    email = body.email.strip().lower()
    if db.scalar(select(User).where(User.email == email)):
        raise HTTPException(409, 'Email already registered')
    u = User(email=email, name=body.name.strip(),
password_hash=hash_password(body.password))
    db.add(u)
    db.commit()
    return {'access_token': make_token(u.id, u.role), 'name': u.name, 'role':
u.role}

@router.post('/login')
def login(form: OAuth2PasswordRequestForm = Depends(), db: Session =
Depends(get_db)):
    u = db.scalar(select(User).where(User.email ==
form.username.strip().lower()))
    if not u or not verify_password(u.password_hash, form.password):
        raise HTTPException(401, 'Wrong email or password')
    return {'access_token': make_token(u.id, u.role), 'token_type': 'bearer',
'name': u.name, 'role': u.role}

```

Add rate limiting on these two routes with the `slowapi` package before you deploy (section 13).

8.3 backend/app/api/reports.py

```
import datetime as dt
import json

from fastapi import APIRouter, BackgroundTasks, Depends, File, Form,
HTTPException, UploadFile
from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from ..core.security import current_user, encrypt_json
from ..db import SessionLocal, get_db
from ..models import HiddenDetail, Match, Place, Report, ReportImage, User
from ..schemas import report_out
from ..services.embeddings import embed_clip_text, embed_image, embed_text
from ..services.imaging import ocr_text, save_image
from ..services.matching import run_matching
from ..services.notify import audit, notify
from ..services.verification import QUESTIONS

router = APIRouter(prefix='/reports', tags=['reports'])

def process_matches(report_id: int):
    db = SessionLocal() # background tasks need their own session
    try:
        for m in run_matching(db, report_id):
            notify(db, m.lost.user_id, 'Potential match found for your lost
item', '/reports/%d' % m.lost_id)
            notify(db, m.found.user_id, 'A possible owner exists for the item
you found', '/reports/%d' % m.found_id)
            db.commit()
    finally:
        db.close()

@router.get('/categories')
def categories():
    return {c: {'group': v['group'], 'questions': {k: q['q'] for k, q in
v['questions'].items()}} for c, v in QUESTIONS.items()}

@router.post('')
async def create_report(
    background: BackgroundTasks,
    type: str = Form(...), category: str = Form(...), brand: str = Form(''),
```

```

color: str = Form(''),
    description: str = Form(...), place_id: int = Form(...), event_time:
dt.datetime = Form(...),
    time_window_hours: float = Form(1.0), hidden_details: str = Form('{}'),
    image: UploadFile | None = File(None),
    db: Session = Depends(get_db), user: User = Depends(current_user),
):
    if type not in ('lost', 'found'):
        raise HTTPException(400, 'type must be lost or found')
    if len(description.strip()) < 10:
        raise HTTPException(400, 'Please describe the item in at least 10
characters')
    if event_time.replace(tzinfo=None) > dt.datetime.utcnow() +
dt.timedelta(hours=24):
        raise HTTPException(400, 'Time cannot be in the future')
    if not db.get(Place, place_id):
        raise HTTPException(404, 'Unknown place')
    group = QUESTIONS[category]['group'] if category in QUESTIONS else
'other'
    rep = Report(user_id=user.id, type=type, category=category, group=group,
brand=brand.strip(),
                color=color.strip().lower(),
description=description.strip(), place_id=place_id,
                event_time=event_time.replace(tzinfo=None),
time_window_hours=time_window_hours)
    rep.txt_emb = embed_text(rep.description)
    rep.clip_txt_emb = embed_clip_text(rep.description)
    db.add(rep)
    db.flush()
    if image is not None and image.filename:
        private, public = save_image(await image.read(), image.content_type)
        rep.img_emb = embed_image(private)
        rep.ocr_text = ocr_text(private)
        db.add(ReportImage(report_id=rep.id, private_path=private,
public_path=public))
    if type == 'found':
        try:
            raw = json.loads(hidden_details)
        except ValueError:
            raise HTTPException(400, 'hidden_details must be JSON')
        allowed = QUESTIONS.get(category, {}).get('questions', {})
        clean = {k: v.strip() for k, v in raw.items() if k in allowed and
isinstance(v, str) and v.strip()}
        if len(clean) < 3:
            raise HTTPException(400, 'Answer at least 3 private questions
about the item')

```

```

        db.add(HiddenDetail(report_id=rep.id, payload=encrypt_json(clean)))
    audit(db, user.id, 'report.create', 'report', rep.id, type=type)
    db.commit()
    background.add_task(process_matches, rep.id)
    return report_out(rep, owner=True)

@router.get('/mine')
def my_reports(db: Session = Depends(get_db), user: User =
    Depends(current_user)):
    rows = db.scalars(select(Report).where(Report.user_id ==
user.id).order_by(Report.created_at.desc())).all()
    return [report_out(r, True) for r in rows]

@router.get('/{rid}')
def get_report(rid: int, db: Session = Depends(get_db), user: User =
    Depends(current_user)):
    r = db.get(Report, rid)
    if not r:
        raise HTTPException(404, 'Not found')
    return report_out(r, owner=(r.user_id == user.id))

@router.get('/{rid}/matches')
def report_matches(rid: int, db: Session = Depends(get_db), user: User =
    Depends(current_user)):
    r = db.get(Report, rid)
    if not r or r.user_id != user.id:
        raise HTTPException(404, 'Not found')
    rows = db.scalars(select(Match).where(or_(Match.lost_id == rid,
Match.found_id == rid), Match.status != 'dismissed')
        .order_by(Match.score.desc())).all()

    out = []
    for m in rows:
        other = m.found if m.lost_id == rid else m.lost
        out.append({'match_id': m.id, 'score': m.score, 'confidence':
m.confidence, 'components': m.components,
                    'status': m.status, 'other': report_out(other)})
    return out

@router.post('/matches/{mid}/dismiss')
def dismiss(mid: int, db: Session = Depends(get_db), user: User =
    Depends(current_user)):
    m = db.get(Match, mid)

```



```

if not m or user.id not in (m.lost.user_id, m.found.user_id):
    raise HTTPException(404, 'Not found')
m.status = 'dismissed'
audit(db, user.id, 'match.dismiss', 'match', m.id)
db.commit()
return {'ok': True}

```

Route order matters: `/categories` and `/mine` are declared before `/ {rid}` so they are not captured as an id. Embeddings run inside an async handler for simplicity, which blocks the server briefly. Before real traffic, wrap the embedding calls with `fastapi.concurrency.run_in_threadpool`.

8.4 backend/app/api/search.py

```

import io

from fastapi import APIRouter, Depends, File, Form, HTTPException, UploadFile
from sqlalchemy import select
from sqlalchemy.orm import Session

from ..core.security import current_user
from ..db import get_db
from ..models import Report, User
from ..schemas import report_out
from ..services.embeddings import embed_clip_text, embed_image, embed_text
from ..services.matching import CROSS_MODAL_SCALE, cosine

router = APIRouter(tags=['search'])

@router.post('/search')
async def search(text: str = Form(''), category: str = Form(''), image:
UploadFile | None = File(None),
                db: Session = Depends(get_db), user: User =
Depends(current_user)):
    has_text = bool(text.strip())
    has_img = image is not None and bool(image.filename)
    if not has_text and not has_img:
        raise HTTPException(400, 'Provide a description, a photo, or both')
    qt = embed_text(text) if has_text else None
    qc = embed_clip_text(text) if has_text else None
    qi = embed_image(io.BytesIO(await image.read())) if has_img else None
    q = select(Report).where(Report.type == 'found', Report.status == 'open')
    if category:
        q = q.where(Report.category == category)

```

```

if qi is not None:
    q = q.order_by(Report.img_emb.cosine_distance(qi))
else:
    q = q.order_by(Report.txt_emb.cosine_distance(qt))
out = []
for r in db.scalars(q.limit(100)).all():
    parts = []
    if qt is not None:
        parts.append(cosine(qt, r.txt_emb))
    if qi is not None:
        parts.append(cosine(qi, r.img_emb) if r.img_emb is not None else
cosine(qi, r.clip_txt_emb) * CROSS_MODAL_SCALE)
    if qc is not None and r.img_emb is not None:
        parts.append(cosine(qc, r.img_emb) * CROSS_MODAL_SCALE)
    parts = [min(1.0, max(0.0, p)) for p in parts if p is not None]
    if parts:
        out.append(**report_out(r), 'score': round(sum(parts) /
len(parts), 4))
    out.sort(key=lambda d: -d['score'])
    return out[:20]

```

Search returns public fields only. A searcher can see that a found item exists, but never its hidden details.

8.5 backend/app/api/claims.py

```

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import select, update
from sqlalchemy.orm import Session

from ..core.config import settings
from ..core.security import current_user, decrypt_json, encrypt_json,
hash_password, verify_password
from ..db import get_db
from ..models import Claim, Match, User, now
from ..services.notify import audit, notify
from ..services.verification import decision_from, new_handover_code,
question_text, score_answers

router = APIRouter(tags=['claims'])

class AnswersIn(BaseModel):
    answers: dict[str, str]

```

```

class CodeIn(BaseModel):
    code: str

def _keys(found):
    return list(decrypt_json(found.hidden.payload))[:3]

@router.post('/matches/{mid}/claim')
def start_claim(mid: int, db: Session = Depends(get_db), user: User =
Depends(current_user)):
    m = db.get(Match, mid)
    if not m:
        raise HTTPException(404, 'Not found')
    if m.lost.user_id != user.id:
        raise HTTPException(403, 'Only the owner of the lost report can start
a claim')
    if m.status in ('dismissed', 'closed') or m.found.status != 'open':
        raise HTTPException(400, 'This match is no longer available')
    c = db.scalar(select(Claim).where(Claim.match_id == mid,
Claim.claimant_id == user.id))
    if c is None:
        c = Claim(match_id=mid, claimant_id=user.id)
        db.add(c)
        db.commit()
    if c.attempts >= settings.MAX_CLAIM_ATTEMPTS or c.decision in
('approved', 'manual_review'):
        raise HTTPException(403, 'No verification attempts left for this
match')
    qs = [{'key': k, 'question': question_text(m.found.category, k)} for k in
_keys(m.found)]
    return {'claim_id': c.id, 'attempts_left': settings.MAX_CLAIM_ATTEMPTS -
c.attempts, 'questions': qs}

@router.post('/claims/{cid}/answers')
def submit_answers(cid: int, body: AnswersIn, db: Session = Depends(get_db),
user: User = Depends(current_user)):
    c = db.get(Claim, cid)
    if not c or c.claimant_id != user.id:
        raise HTTPException(404, 'Not found')
    if c.attempts >= settings.MAX_CLAIM_ATTEMPTS or c.decision in
('approved', 'manual_review'):
        raise HTTPException(403, 'No verification attempts left')
    m, found = c.match, c.match.found

```

```

        hidden = decrypt_json(found.hidden.payload)
        keys = _keys(found)
        result = score_answers(found.category, {k: hidden[k] for k in keys},
body.answers)
        n = sum(result.values())
        decision = decision_from(n, len(keys))
        c.attempts += 1
        c.n_correct = n
        c.decision = decision
        c.answers = encrypt_json(body.answers)
        code = None
        if decision == 'approved':
            code = new_handover_code()
            c.handover_hash = hash_password(code)
            m.status = 'claimed'
            found.status = 'claimed'
            notify(db, found.user_id, 'An owner passed verification. Arrange the
handover and ask for their code.', '/reports/%d' % found.id)
            audit(db, user.id, 'claim.answers', 'claim', c.id, n_correct=n,
decision=decision)
            db.commit()
            label = {'approved': 'approved', 'manual_review': 'under_review',
'rejected': 'not_verified'}[decision]
            return {'result': label, 'handover_code': code, 'attempts_left':
settings.MAX_CLAIM_ATTEMPTS - c.attempts}

@router.post('/claims/{cid}/confirm')
def confirm_handover(cid: int, body: CodeIn, db: Session = Depends(get_db),
user: User = Depends(current_user)):
    c = db.get(Claim, cid)
    if not c or c.match_found.user_id != user.id:
        raise HTTPException(404, 'Not found')
    if c.decision != 'approved' or not c.handover_hash:
        raise HTTPException(400, 'Claim is not approved')
    if not verify_password(c.handover_hash, body.code.strip()):
        audit(db, user.id, 'handover.wrong_code', 'claim', c.id)
        db.commit()
        raise HTTPException(400, 'Wrong code')
    m = c.match
    for r in (m.lost, m.found):
        r.status = 'closed'
        r.closed_at = now()
    m.status = 'closed'
    db.execute(update(Match).where((Match.lost_id == m.lost_id) |
(Match.found_id == m.found_id),

```

```

        Match.id != m.id, Match.status ==
'suggested').values(status='dismissed'))
    notify(db, m.lost.user_id, 'Handover confirmed. Your report is closed.',
'/reports/%d' % m.lost_id)
    audit(db, user.id, 'handover.confirm', 'claim', c.id)
    db.commit()
    return {'ok': True}

```

The response never says how many answers were right, so a claimant cannot tune guesses between attempts. The handover code is shown once, to the claimant, and only its hash is stored.

8.6 backend/app/main.py

```

import os

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from sqlalchemy import text

from .api import admin, auth, claims, misc, reports, search
from .core.config import settings
from .db import engine
from .models import Base

os.makedirs(settings.UPLOAD_DIR + '/public', exist_ok=True)

app = FastAPI(title='Lost and Found Intelligence')
app.add_middleware(CORSMiddleware,
allow_origins=settings.CORS_ORIGINS.split(','),
                    allow_credentials=True, allow_methods=['*'],
allow_headers=['*'])
app.mount('/files/public', StaticFiles(directory=settings.UPLOAD_DIR +
'/public'), name='files')
for module in (auth, reports, search, claims, admin, misc):
    app.include_router(module.router)

@app.on_event('startup')
def startup():
    with engine.begin() as conn:
        conn.execute(text('CREATE EXTENSION IF NOT EXISTS vector'))
    Base.metadata.create_all(engine)

```

```
@app.get('/health')
def health():
    return {'status': 'ok'}
```

Only the public folder is mounted. The private originals are never reachable over HTTP.

8.7 backend/app/api/admin.py

```
import statistics

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from ..core.security import decrypt_json, hash_password, require_moderator
from ..db import get_db
from ..models import Claim, Place, Report, User
from ..schemas import report_out
from ..services.notify import audit, notify
from ..services.verification import new_handover_code, question_text

router = APIRouter(prefix='/admin', tags=['admin'])

class DecisionIn(BaseModel):
    decision: str # approved | rejected

@router.get('/queue')
def queue(db: Session = Depends(get_db), mod: User =
Depends(require_moderator)):
    claims = db.scalars(select(Claim).where(Claim.decision ==
'manual_review').order_by(Claim.created_at)).all()
    out = []
    for c in claims:
        found = c.match.found
        hidden = decrypt_json(found.hidden.payload)
        given = decrypt_json(c.answers) if c.answers else {}
        rows = [{'question': question_text(found.category, k),
'finder_detail': hidden[k], 'claimant_answer': given.get(k, '')}
                for k in list(hidden)[:3]]
        out.append({'claim_id': c.id, 'found': report_out(found), 'lost':
report_out(c.match.lost), 'rows': rows})
        audit(db, mod.id, 'queue.view', 'claim', c.id)
```

```

db.commit()
return out

@router.post('/claims/{cid}/decision')
def decide(cid: int, body: DecisionIn, db: Session = Depends(get_db), mod:
User = Depends(require_moderator)):
    c = db.get(Claim, cid)
    if not c or c.decision != 'manual_review':
        raise HTTPException(404, 'Not found')
    if body.decision not in ('approved', 'rejected'):
        raise HTTPException(400, 'decision must be approved or rejected')
    c.decision, c.decided_by = body.decision, mod.id
    if body.decision == 'approved':
        code = new_handover_code()
        c.handover_hash = hash_password(code)
        c.match.status = c.match.found.status = 'claimed'
        notify(db, c.claimant_id, 'Verification approved. Your handover code
is %s. Show it to the finder.' % code)
        notify(db, c.match.found.user_id, 'An owner was approved. Arrange the
handover and ask for their code.')
    else:
        notify(db, c.claimant_id, 'Your claim could not be verified.')
    audit(db, mod.id, 'claim.decision', 'claim', c.id,
decision=body.decision)
    db.commit()
    return {'ok': True}

@router.get('/analytics')
def analytics(db: Session = Depends(get_db), mod: User =
Depends(require_moderator)):
    lost_total = db.scalar(select(func.count()).where(Report.type == 'lost'))
or 0
    lost_closed = db.scalar(select(func.count()).where(Report.type == 'lost',
Report.status == 'closed')) or 0
    spans = db.execute(select(Report.created_at,
Report.closed_at).where(Report.type == 'lost',
Report.closed_at.is_not(None))).all()
    hours = [(b - a).total_seconds() / 3600 for a, b in spans]
    by_cat = db.execute(select(Report.category, Report.type,
func.count()).group_by(Report.category, Report.type)).all()
    hotspots = db.execute(select(Place.name, func.count()).join(Report,
Report.place_id == Place.id)
                        .where(Report.type ==
'lost').group_by(Place.name).order_by(func.count().desc())).all()

```

```

    daily = db.execute(select(func.date(Report.created_at), Report.type,
func.count())
                        .group_by(func.date(Report.created_at),
Report.type).order_by(func.date(Report.created_at))).all()
    return {
        'recovery_rate': round(lost_closed / lost_total, 3) if lost_total
else 0,
        'median_hours_to_recovery': round(statistics.median(hours), 1) if
hours else None,
        'by_category': [{'category': c, 'type': t, 'count': n} for c, t, n in
by_cat],
        'hotspots': [{'place': p, 'count': n} for p, n in hotspots],
        'daily': [{'date': str(d), 'type': t, 'count': n} for d, t, n in
daily],
    }

```

8.8 backend/app/api/misc.py

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from ..core.security import current_user
from ..db import get_db
from ..models import AbuseReport, Notification, Place, User

router = APIRouter(tags=['misc'])

@router.get('/places')
def places(db: Session = Depends(get_db)):
    return [{'id': p.id, 'name': p.name, 'latitude': p.latitude, 'longitude':
p.longitude}
            for p in db.scalars(select(Place).order_by(Place.name)).all()]

@router.get('/notifications')
def notifications(db: Session = Depends(get_db), user: User =
Depends(current_user)):
    rows = db.scalars(select(Notification).where(Notification.user_id ==
user.id)

.order_by(Notification.created_at.desc()).limit(50)).all()
    return [{'id': n.id, 'text': n.text, 'link': n.link, 'read': n.read,
'created_at': n.created_at.isoformat()} for n in rows]

```



```

@router.post('/notifications/{nid}/read')
def mark_read(nid: int, db: Session = Depends(get_db), user: User =
Depends(current_user)):
    n = db.get(Notification, nid)
    if not n or n.user_id != user.id:
        raise HTTPException(404, 'Not found')
    n.read = True
    db.commit()
    return {'ok': True}

@router.post('/reports/{rid}/abuse')
def report_abuse(rid: int, reason: str, db: Session = Depends(get_db), user:
User = Depends(current_user)):
    db.add(AbuseReport(reporter_id=user.id, report_id=rid,
reason=reason[:300]))
    db.commit()
    return {'ok': True}

```

9. Seeding and first run

Unzip `lost_found_dataset.zip` into the `data/` folder so the CSVs sit in `data/lost_found_dataset/`. The seed script loads the places, creates demo users, loads the test-split reports (about 220) with real embeddings, and runs the matcher so the demo has matches from the start.

`scripts/seed.py`

```

import json
import sys

import pandas as pd
from sqlalchemy import select, text

sys.path.insert(0, '/code')
from app.core.security import encrypt_json, hash_password
from app.db import SessionLocal, engine
from app.models import Base, HiddenDetail, Place, Report, User
from app.services.embeddings import embed_clip_text, embed_text
from app.services.matching import run_matching
from app.services.verifications import QUESTIONS

DATA = '/data/seed/lost_found_dataset/'
SPLIT = sys.argv[1] if len(sys.argv) > 1 else 'test'

```

```

COLORS = ['navy blue', 'black', 'grey', 'gray', 'red', 'green', 'maroon',
'blue', 'white', 'silver', 'brown', 'pink', 'orange']

def guess_color(desc):
    d = desc.lower()
    return next((c.replace('gray', 'grey') for c in COLORS if c in d), '')

with engine.begin() as conn:
    conn.execute(text('CREATE EXTENSION IF NOT EXISTS vector'))
Base.metadata.create_all(engine)
db = SessionLocal()

for r in pd.read_csv(DATA + 'campus_places.csv').itertuples():
    if not db.scalar(select(Place).where(Place.name == r.place)):
        db.add(Place(name=r.place, latitude=r.latitude,
longitude=r.longitude))
db.commit()
place_id = {p.name: p.id for p in db.scalars(select(Place)).all()}

if not db.scalar(select(User).where(User.email == 'admin@demo.test')):
    db.add(User(email='admin@demo.test', name='Moderator', role='moderator',
password_hash=hash_password('admin12345')))
    db.commit()

lost = pd.read_csv(DATA + 'lost_reports.csv')
found = pd.read_csv(DATA + 'found_reports.csv')
items = pd.read_csv(DATA + 'items_ground_truth.csv').set_index('item_id')
lost, found = lost[lost.split == SPLIT], found[found.split == SPLIT]

user_id = {}
for uid in sorted(set(lost.user_id) | set(found.user_id)):
    u = User(email=uid.lower() + '@demo.test', name=uid,
password_hash=hash_password('demo1234'))
    db.add(u)
    db.flush()
    user_id[uid] = u.id
db.commit()

lost_ids = []
for kind, df in (('lost', lost), ('found', found)):
    for r in df.itertuples():
        # found reports use the true category so the private questions match
the item
        cat = items.loc[r.item_id, 'category'] if kind == 'found' else

```

```

r.category_reported
    group = QUESTIONS[cat]['group'] if cat in QUESTIONS else 'other'
    when = pd.to_datetime(r.lost_time_reported if kind == 'lost' else
r.found_time).to_pydatetime()
    rep = Report(user_id=user_id[r.user_id], type=kind, category=cat,
group=group, color=guess_color(r.description),
                description=r.description,
place_id=place_id[r.place_reported], event_time=when,
                time_window_hours=float(getattr(r, 'time_window_hours',
1.0) or 1.0),
                txt_emb=embed_text(r.description),
clip_txt_emb=embed_clip_text(r.description))
    db.add(rep)
    db.flush()
    if kind == 'found':
        hidden = json.loads(r.hidden_details_json)
        db.add(HiddenDetail(report_id=rep.id,
payload=encrypt_json(hidden)))
    else:
        lost_ids.append(rep.id)
    db.commit()

for rid in lost_ids:
    run_matching(db, rid)
print('seeded', len(lost), 'lost and', len(found), 'found reports')

```

First run checklist

1. Copy the data and generate the question file on your machine from the project root:
`python scripts/export_questions.py`. The backend folder is mounted into the container, so `questions.json` appears there automatically.
2. `docker compose up --build -d`
3. `docker compose exec backend python scripts/seed.py`
4. Open `http://localhost:8000/docs` and try register, login, then `GET /places`.
5. Log in as `u0001@demo.test` with password `demo1234`, or as `admin@demo.test` with password `admin12345` for the moderator view. Change these credentials before any public deployment.

10. Frontend (React, Vite, Tailwind)

10.1 Scaffold

Run this in the project root, then add the Dockerfile from section 4 into `frontend/`.

```
npm create vite@latest frontend -- --template react
cd frontend
npm install react-router-dom axios recharts
npm install -D tailwindcss@3 postcss autoprefixer
npx tailwindcss init -p
```

tailwind.config.js: set `content` to `['./index.html', './src/**/*.{js,jsx}']`.

src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

vite.config.js

```
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vite';

export default defineConfig({ plugins: [react()], server: { host: true, port: 5173 } });
```

src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
```

10.2 src/api.js

```
import axios from 'axios';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL ||
'http://localhost:8000' });

api.interceptors.request.use((cfg) => {
  const t = localStorage.getItem('token');
  if (t) cfg.headers.Authorization = 'Bearer ' + t;
  return cfg;
});

export const API = api.defaults.baseURL;
export const errMsg = (ex, fallback) => {
  const d = ex.response?.data?.detail;
  return typeof d === 'string' ? d : fallback;
};
export default api;
```

Storing the token in `localStorage` is fine for a project. For a production system, prefer an `httpOnly` cookie.

10.3 src/App.jsx

```
import { BrowserRouter, Link, Navigate, Route, Routes, useNavigate } from
'react-router-dom';
import Admin from './pages/Admin';
import ClaimPage from './pages/ClaimPage';
import Home from './pages/Home';
import Login from './pages/Login';
import MyReports from './pages/MyReports';
import Notifications from './pages/Notifications';
import ReportDetail from './pages/ReportDetail';
import ReportForm from './pages/ReportForm';

function Private({ children }) {
  return localStorage.getItem('token') ? children : <Navigate to='/login' />;
}

function Nav() {
  const nav = useNavigate();
  const token = localStorage.getItem('token');
  const role = localStorage.getItem('role');
  const logout = () => {
    localStorage.clear();
```

```

    nav('/login');
  };
  return (
    <nav className='flex flex-wrap gap-4 items-center p-3 bg-slate-900 text-white'>
      <Link to='/' className='font-bold'>Lost and Found</Link>
      {token && (
        <>
          <Link to='/report/lost'>Report lost</Link>
          <Link to='/report/found'>Report found</Link>
          <Link to='/mine'>My reports</Link>
          <Link to='/notifications'>Notifications</Link>
          {role === 'moderator' && <Link to='/admin'>Moderator</Link>}
          <button onClick={logout} className='ml-auto'>Log out</button>
        </>
      )}
    </nav>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Nav />
      <Routes>
        <Route path='/login' element={<Login />} />
        <Route path='/' element={<Private><Home /></Private>} />
        <Route path='/report/:type' element={<Private><ReportForm />
</Private>} />
        <Route path='/mine' element={<Private><MyReports /></Private>} />
        <Route path='/reports/:id' element={<Private><ReportDetail />
</Private>} />
        <Route path='/claim/:matchId' element={<Private><ClaimPage />
</Private>} />
        <Route path='/notifications' element={<Private><Notifications />
</Private>} />
        <Route path='/admin' element={<Private><Admin /></Private>} />
      </Routes>
    </BrowserRouter>
  );
}

```

10.4 src/pages/Login.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import api, { errMsg } from '../api';

export default function Login() {
  const [mode, setMode] = useState('login');
  const [f, setF] = useState({ email: '', password: '', name: '' });
  const [err, setErr] = useState('');
  const nav = useNavigate();
  const set = (k) => (e) => setF({ ...f, [k]: e.target.value });

  const submit = async (e) => {
    e.preventDefault();
    setErr('');
    try {
      const res = mode === 'login'
        ? await api.post('/auth/login', new URLSearchParams({ username:
f.email, password: f.password }))
        : await api.post('/auth/register', f);
      localStorage.setItem('token', res.data.access_token);
      localStorage.setItem('role', res.data.role);
      nav('/');
    } catch (ex) {
      setErr(errMsg(ex, 'Something went wrong'));
    }
  };

  return (
    <form onSubmit={submit} className='max-w-sm mx-auto mt-10 space-y-3 p-4'>
      <h1 className='text-xl font-bold'>{mode === 'login' ? 'Log in' :
'Create account'}</h1>
      {mode === 'register' && <input className='border rounded p-2 w-full'
placeholder='Name' value={f.name} onChange={set('name')} />}
      <input className='border rounded p-2 w-full' placeholder='Email' value=
{f.email} onChange={set('email')} />
      <input className='border rounded p-2 w-full' type='password'
placeholder='Password (8+ characters)' value={f.password} onChange=
{set('password')} />
      {err && <p className='text-red-600 text-sm'>{err}</p>}
      <button className='bg-blue-600 text-white rounded px-4 py-2 w-full'>
{mode === 'login' ? 'Log in' : 'Register'}</button>
      <button type='button' className='text-sm underline' onClick={() =>
setMode(mode === 'login' ? 'register' : 'login')}>
        {mode === 'login' ? 'Need an account? Register' : 'Have an account?

```

```

    Log in'}
      </button>
    </form>
  );
}

```

10.5 src/pages/Home.jsx (multimodal search)

```

import { useState } from 'react';
import api, { API, errMsg } from '../api';

export default function Home() {
  const [text, setText] = useState('');
  const [image, setImage] = useState(null);
  const [rows, setRows] = useState(null);
  const [err, setErr] = useState('');

  const go = async (e) => {
    e.preventDefault();
    setErr('');
    const fd = new FormData();
    fd.append('text', text);
    if (image) fd.append('image', image);
    try {
      setRows((await api.post('/search', fd)).data);
    } catch (ex) {
      setErr(errMsg(ex, 'Search failed'));
    }
  };

  return (
    <div className='max-w-3xl mx-auto p-4'>
      <h1 className='text-2xl font-bold mb-3'>Search found items</h1>
      <form onSubmit={go} className='space-y-2'>
        <textarea className='border rounded p-2 w-full' rows={3} value={text}
onChange={(e) => setText(e.target.value)}
placeholder='Small blue water bottle with a silver cap' />
        <input type='file' accept='image/*' onChange={(e) =>
setImage(e.target.files[0])} />
        <button className='bg-blue-600 text-white px-4 py-2 rounded
block'>Search</button>
      </form>
      {err && <p className='text-red-600'>{err}</p>}
      {rows && rows.length === 0 && <p className='mt-4'>No found items match
yet.</p>}
      <div className='grid gap-3 mt-4'>

```



```

    {rows?.map((r) => (
      <div key={r.id} className='border rounded p-3 flex gap-3'>
        {r.images[0] && <img src={API + r.images[0]} className='w-24 h-24
object-cover rounded' />}
        <div>
          <div className='font-semibold'>{r.color} {r.category}
            <span className='text-xs text-gray-500 ml-2'>similarity
{Math.round(r.score * 100)}%</span></div>
          <div className='text-sm'>{r.description}</div>
          <div className='text-xs text-gray-500'>{r.place} at {new
Date(r.event_time).toLocaleString()}</div>
        </div>
      </div>
    ))}
  </div>
</div>
);
}

```

10.6 src/pages/ReportForm.jsx

For a found report the form shows the private questions for the chosen category. Those answers are the hidden details used later for verification.

```

import { useEffect, useState } from 'react';
import { useNavigate, useParams } from 'react-router-dom';
import api, { errMsg } from '../api';

const box = 'border rounded p-2 w-full';

export default function ReportForm() {
  const { type } = useParams();
  const nav = useNavigate();
  const [places, setPlaces] = useState([]);
  const [cats, setCats] = useState({});
  const [f, setF] = useState({ category: '', brand: '', color: '',
description: '', place_id: '', event_time: '', time_window_hours: 1 });
  const [hidden, setHidden] = useState({});
  const [image, setImage] = useState(null);
  const [err, setErr] = useState('');

  useEffect(() => {
    api.get('/places').then((r) => setPlaces(r.data));
    api.get('/reports/categories').then((r) => setCats(r.data));
  }, []);

```

```

const set = (k) => (e) => setF({ ...f, [k]: e.target.value });
const questions = cats[f.category]?.questions || {};

const submit = async (e) => {
  e.preventDefault();
  setErr('');
  const fd = new FormData();
  Object.entries({ ...f, type }).forEach(([k, v]) => fd.append(k, v));
  if (type === 'found') fd.append('hidden_details',
JSON.stringify(hidden));
  if (image) fd.append('image', image);
  try {
    await api.post('/reports', fd);
    nav('/mine');
  } catch (ex) {
    setErr(errMsg(ex, 'Could not save the report. Check all fields.'));
  }
};

return (
  <form onSubmit={submit} className='max-w-xl mx-auto p-4 space-y-3'>
    <h1 className='text-xl font-bold'>Report a {type} item</h1>
    <select className={box} value={f.category} onChange={set('category')}
required>
      <option value=''>Category</option>
      {Object.keys(cats).map((c) => <option key={c}>{c}</option>)}
    </select>
    <div className='flex gap-2'>
      <input className={box} placeholder='Brand (optional)' value={f.brand}
onChange={set('brand')} />
      <input className={box} placeholder='Main colour' value={f.color}
onChange={set('color')} />
    </div>
    <textarea className={box} rows={3} required placeholder='Describe the
item (no private details)' value={f.description} onChange=
{set('description')} />
    <select className={box} value={f.place_id} onChange={set('place_id')}
required>
      <option value=''>Place</option>
      {places.map((p) => <option key={p.id} value={p.id}>{p.name}
</option>)}
    </select>
    <input className={box} type='datetime-local' required value=
{f.event_time} onChange={set('event_time')} />
    {type === 'lost' && (
      <select className={box} value={f.time_window_hours} onChange=

```

```

{set('time_window_hours')}}>
    <option value={0.5}>Within the last 30 minutes of that
time</option>
    <option value={1}>Within about 1 hour</option>
    <option value={2}>Within about 2 hours</option>
    <option value={4}>Within about 4 hours</option>
  </select>
})
  <input type='file' accept='image/*' onChange={(e) =>
setImage(e.target.files[0])} />
  {type === 'found' && Object.keys(questions).length > 0 && (
    <div className='border rounded p-3 space-y-2 bg-amber-50'>
      <p className='text-sm'>Answer at least 3. These stay private and
are used only to verify the real owner.</p>
      {Object.entries(questions).map(([k, q]) => (
        <input key={k} className={box} placeholder={q} value={hidden[k]
|| ''} onChange={(e) => setHidden({ ...hidden, [k]: e.target.value })} />
      ))}
    </div>
  )}
  {err && <p className='text-red-600 text-sm'>{err}</p>}
  <button className='bg-blue-600 text-white rounded px-4 py-
2'>Submit</button>
</form>
);
}

```

10.7 src/pages/MyReports.jsx and Notifications.jsx

```
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import api from '../api';

export default function MyReports() {
  const [rows, setRows] = useState([]);
  useEffect(() => { api.get('/reports/mine').then((r) => setRows(r.data)); },
    []);
  return (
    <div className='max-w-2xl mx-auto p-4 space-y-2'>
      <h1 className='text-xl font-bold'>My reports</h1>
      {rows.map((r) => (
        <Link key={r.id} to={'/reports/' + r.id} className='block border
rounded p-3 hover:bg-gray-50'>
          <b>{r.type.toUpperCase()}</b> {r.color} {r.category} at {r.place}
          <span className='float-right text-xs'>{r.status}</span>
        </Link>
      ))}
    </div>
  );
}
```

Notifications.jsx follows the same pattern: `api.get('/notifications')`, render each item as a link to `n.link`, and call `api.post('/notifications/' + n.id + '/read')` when it is clicked.

10.8 src/pages/ReportDetail.jsx (matches, score breakdown, handover)

First, patch `report_matches` in `backend/app/api/reports.py` so the finder can see the claim id once an owner is approved. Import `Claim` from `..models`, then replace the loop body with:

```
for m in rows:
    other = m.found if m.lost_id == rid else m.lost
    claim = db.scalar(select(Claim).where(Claim.match_id == m.id,
Claim.decision == 'approved'))
    out.append({'match_id': m.id, 'score': m.score, 'confidence':
m.confidence, 'components': m.components,
               'status': m.status, 'claim_id': claim.id if claim else
None, 'other': report_out(other)})
```

```

import { useEffect, useState } from 'react';
import { Link, useParams } from 'react-router-dom';
import api, { API, errMsg } from '../api';

const BADGE = { high: 'bg-green-600', medium: 'bg-amber-500', low: 'bg-gray-500' };
const LABELS = { text: 'Description', image: 'Photo', color: 'Colour', brand: 'Brand', geo: 'Place', time: 'Time', ocr: 'Text on item' };

function Bar({ k, v }) {
  if (v == null) return null;
  const w = Math.round(Math.max(0, Math.min(1, v)) * 100) + '%';
  return (
    <div className='flex items-center gap-2 text-xs'>
      <span className='w-24'>{LABELS[k]}</span>
      <div className='h-2 bg-gray-200 flex-1 rounded'><div className='h-2 bg-blue-500 rounded' style={{ width: w }} /></div>
    </div>
  );
}

export default function ReportDetail() {
  const { id } = useParams();
  const [r, setR] = useState(null);
  const [ms, setMs] = useState([]);
  const [code, setCode] = useState('');
  const [msg, setMsg] = useState('');

  const load = () => {
    api.get('/reports/' + id).then((x) => setR(x.data));
    api.get('/reports/' + id + '/matches').then((x) => setMs(x.data)).catch(() => setMs([]));
  };
  useEffect(load, [id]);

  const dismiss = async (mid) => { await api.post('/reports/matches/' + mid + '/dismiss'); load(); };
  const confirm = async (cid) => {
    try {
      await api.post('/claims/' + cid + '/confirm', { code });
      setMsg('Handover confirmed. Thank you!');
      load();
    } catch (ex) {
      setMsg(errMsg(ex, 'Could not confirm'));
    }
  };
}

```

```

if (!r) return <p className='p-4'>Loading...</p>;
return (
  <div className='max-w-2xl mx-auto p-4 space-y-3'>
    <h1 className='text-xl font-bold'>{r.type.toUpperCase()}: {r.color}
    {r.brand} {r.category}</h1>
    <p>{r.description}</p>
    <p className='text-sm text-gray-500'>{r.place} at {new
Date(r.event_time).toLocaleString()} (status: {r.status})</p>
    <h2 className='font-semibold pt-2'>Potential matches</h2>
    {ms.length === 0 && <p className='text-sm'>No suggestions yet. We will
notify you.</p>}
    {ms.map((m) => (
      <div key={m.match_id} className='border rounded p-3 space-y-2'>
        <div className='flex gap-3'>
          {m.other.images[0] && <img src={API + m.other.images[0]}
className='w-24 h-24 object-cover rounded' />}
          <div className='flex-1'>
            <span className={`text-white text-xs px-2 py-1 rounded ` +
BADGE[m.confidence]}>{m.confidence} confidence</span>
            <p className='text-sm mt-1'>{m.other.description}</p>
            <p className='text-xs text-gray-500'>{m.other.place} at {new
Date(m.other.event_time).toLocaleString()}</p>
          </div>
        </div>
        {Object.entries(m.components).map(([k, v]) => <Bar key={k} k={k} v=
{v} />)}
        <div className='flex gap-2'>
          {r.type === 'lost' && m.status === 'suggested' && (
            <Link to={`/${claim}/` + m.match_id} className='bg-blue-600 text-
white rounded px-3 py-1 text-sm'>Verify ownership</Link>
          )}
          {m.status === 'suggested' && <button className='text-sm
underline' onClick={() => dismiss(m.match_id)}>Not a match</button>}
        </div>
        {r.type === 'found' && m.status === 'claimed' && m.claim_id && (
          <div className='flex gap-2'>
            <input className='border rounded p-1' placeholder='Owner
handover code' value={code} onChange={(e) => setCode(e.target.value)} />
            <button className='bg-green-600 text-white rounded px-3 py-1
text-sm' onClick={() => confirm(m.claim_id)}>Confirm handover</button>
          </div>
        )}
      </div>
    ))}
  </div>
))}

```

```

    {msg && <p className='text-sm'>{msg}</p>}
  </div>
);
}

```

The wording is deliberate: the UI says potential match and confidence. It never says this is your item.

10.9 src/pages/ClaimPage.jsx

```

import { useEffect, useState } from 'react';
import { Link, useParams } from 'react-router-dom';
import api, { errMsg } from '../api';

export default function ClaimPage() {
  const { matchId } = useParams();
  const [claim, setClaim] = useState(null);
  const [answers, setAnswers] = useState({});
  const [result, setResult] = useState(null);
  const [err, setErr] = useState('');

  useEffect(() => {
    api.post('/matches/' + matchId + '/claim').then((r) => setClaim(r.data))
      .catch((ex) => setErr(errMsg(ex, 'Cannot start verification')));
  }, [matchId]);

  const submit = async () => {
    setErr('');
    try {
      setResult((await api.post('/claims/' + claim.claim_id + '/answers', {
answers })).data);
    } catch (ex) {
      setErr(errMsg(ex, 'Could not submit'));
    }
  };

  if (err && !claim) return <p className='p-4 text-red-600'>{err}</p>;
  if (!claim) return <p className='p-4'>Loading...</p>;
  if (result) {
    return (
      <div className='max-w-xl mx-auto p-4 space-y-3'>
        {result.result === 'approved' && (
          <>
            <h1 className='text-xl font-bold text-green-700'>Ownership
verified</h1>
            <p>Show this one-time code to the finder at handover. It will not

```

```

be shown again.</p>
      <p className='text-4xl font-mono tracking-widest'>
{result.handover_code}</p>
    </>
  )}
  {result.result === 'under_review' && <p>A moderator will review your
answers. You will get a notification.</p>}
  {result.result === 'not_verified' && <p>We could not verify these
answers. Attempts left: {result.attempts_left}</p>}
    <Link to='/mine' className='underline'>Back to my reports</Link>
  </div>
);
}
return (
  <div className='max-w-xl mx-auto p-4 space-y-3'>
    <h1 className='text-xl font-bold'>Ownership verification</h1>
    <p className='text-sm'>Only the real owner can answer these. Attempts
left: {claim.attempts_left}</p>
    {claim.questions.map((q) => (
      <div key={q.key}>
        <label className='block text-sm font-medium'>{q.question}</label>
        <textarea className='border rounded p-2 w-full' rows={2} value=
{answers[q.key] || ''}
          onChange={(e) => setAnswers({ ...answers, [q.key]: e.target.value
        })} />
      </div>
    ))}
    {err && <p className='text-red-600 text-sm'>{err}</p>}
    <button className='bg-blue-600 text-white rounded px-4 py-2' onClick=
{submit}>Submit verification</button>
  </div>
);
}

```

10.10 src/pages/Admin.jsx (moderator queue and analytics)

```

import { useEffect, useState } from 'react';
import { Bar, BarChart, ResponsiveContainer, Tooltip, XAxis, YAxis } from
'recharts';
import api from '../api';

export default function Admin() {
  const [queue, setQueue] = useState([]);
  const [stats, setStats] = useState(null);
  const load = () => {
    api.get('/admin/queue').then((r) => setQueue(r.data)).catch(() =>

```



```

setQueue([]));
  api.get('/admin/analytics').then((r) => setStats(r.data)).catch(() =>
setStats(null));
};
useEffect(load, []);
const decide = async (id, decision) => { await api.post('/admin/claims/' +
id + '/decision', { decision }); load(); };

return (
  <div className='max-w-4xl mx-auto p-4 space-y-6'>
    {stats && (
      <section>
        <h1 className='text-xl font-bold mb-2'>Analytics</h1>
        <div className='flex gap-4 mb-3'>
          <div className='border rounded p-3'>Recovery rate<div
className='text-2xl font-bold'>{Math.round(stats.recovery_rate * 100)}%</div>
</div>
          <div className='border rounded p-3'>Median hours to recovery<div
className='text-2xl font-bold'>{stats.median_hours_to_recovery ?? '-'}</div>
</div>
        </div>
        <h2 className='font-semibold'>Where items are lost</h2>
        <div style={{ height: 260 }}>
          <ResponsiveContainer>
            <BarChart data={stats.hotspots}>
              <XAxis dataKey='place' interval={0} angle={-30}
textAnchor='end' height={80} tick={{ fontSize: 11 }} />
              <YAxis allowDecimals={false} />
              <Tooltip />
              <Bar dataKey='count' fill='#2563eb' />
            </BarChart>
          </ResponsiveContainer>
        </div>
      </section>
    )}
    <section>
      <h1 className='text-xl font-bold mb-2'>Claims needing review
({queue.length})</h1>
      {queue.map((c) => (
        <div key={c.claim_id} className='border rounded p-3 mb-3 space-y-
2'>
          <p className='text-sm'>Found: {c.found.description}</p>
          <table className='text-sm w-full'>
            <tbody>
              {c.rows.map((x) => (
                <tr key={x.question} className='border-t'>

```

```

        <td className='pr-2'>{x.question}</td>
        <td className='pr-2 text-green-700'>{x.finder_detail}
    </td>

        <td>{x.claimant_answer}</td>
    </tr>
    )})
    </tbody>
</table>
    <button className='bg-green-600 text-white rounded px-3 py-1 mr-2' onClick={() => decide(c.claim_id, 'approved')}>Approve</button>
    <button className='bg-red-600 text-white rounded px-3 py-1'
onClick={() => decide(c.claim_id, 'rejected')}>Reject</button>
    </div>
    )})
</section>
</div>
);
}

```

The analytics block is the part to polish for your data-analyst portfolio. Good additions are a daily trend line, recovery rate by category, and a campus hotspot map.

11. Evaluation and the learned ranker

These scripts run against the CSV dataset, not the live database, so you can iterate on the matcher quickly. They reuse the exact scoring code from section 7.4, so the numbers you report describe the code you ship.

11.1 Patch backend/app/services/matching.py

Add this after `weighted_score`, then add the startup hook below to `main.py`.

```

FEATURES = ['text', 'image', 'color', 'brand', 'geo', 'time', 'ocr']

def load_ranker(path):
    import lightgbm as lgb
    booster = lgb.Booster(model_file=path)

    def predict(comp):
        x = [[-1.0 if comp.get(k) is None else comp[k] for k in FEATURES]]
        return 1 / (1 + math.exp(-booster.predict(x)[0])) # squash ranker
        output to 0..1

    return predict

```

```
# backend/app/main.py, inside startup()
import os
from .services import matching
path = os.path.join(os.path.dirname(__file__), 'data', 'ranker.txt')
if os.path.exists(path):
    matching.RANKER = matching.load_ranker(path)
```

After the ranker is switched on, recalibrate the `confidence` thresholds, because the score scale changes.

11.2 scripts/features.py (shared helpers)

```
import sys
from types import SimpleNamespace as NS

import pandas as pd

sys.path.insert(0, '/code')
from app.services.embeddings import embed_text
from app.services.matching import FEATURES, pair_components

DATA = '/data/seed/lost_found_dataset/'
places = pd.read_csv(DATA + 'campus_places.csv').set_index('place')
L = pd.read_csv(DATA + 'lost_reports.csv').set_index('report_id')
F = pd.read_csv(DATA + 'found_reports.csv').set_index('report_id')
COLORS = ['navy blue', 'black', 'grey', 'gray', 'red', 'green', 'maroon',
'blue', 'white', 'silver', 'brown', 'pink', 'orange']
_cache = {}

def guess_color(desc):
    d = desc.lower()
    return next((c.replace('gray', 'grey') for c in COLORS if c in d), '')

def get(kind, rid):
    key = (kind, rid)
    if key not in _cache:
        row = (L if kind == 'lost' else F).loc[rid]
        p = places.loc[row['place_reported']]
        when = row['lost_time_reported'] if kind == 'lost' else
row['found_time']
        window = row['time_window_hours'] if kind == 'lost' else 1.0
        _cache[key] = NS(place=NS(latitude=p['latitude'],
longitude=p['longitude']),
```

```

        event_time=pd.to_datetime(when).to_pydatetime(),
time_window_hours=float(window),
        txt_emb=embed_text(row['description']),
img_emb=None, clip_txt_emb=None,
        color=guess_color(row['description']), brand='',
ocr_text='')
    return _cache[key]

def vector(comp):
    return [-1.0 if comp[k] is None else comp[k] for k in FEATURES]

```

11.3 scripts/evaluate.py (retrieval and ablations)

For each lost report in the test split that has a true match, rank every found report in the same split and record where the true one lands.

```

import os

import numpy as np
import pandas as pd
from features import DATA, F, get, pair_components

from app.services import matching

M = pd.read_csv(DATA + 'matches_ground_truth.csv')
M = M[M.split == 'test']
truth = dict(zip(M.lost_id, M.found_id))
found_ids = list(F[F.split == 'test'].index)
comps = {lid: [pair_components(get('lost', lid), get('found', f)) for f in
found_ids] for lid in truth}

def report(name, scorer):
    ranks = []
    for lid, fid in truth.items():
        s = np.array([scorer(c) for c in comps[lid]])
        order = [found_ids[i] for i in np.argsort(-s)]
        ranks.append(order.index(fid) + 1)
    r = np.array(ranks)
    print('%-26s R@1 %.2f  R@5 %.2f  MRR %.3f' % (name, (r == 1).mean(), (r
<= 5).mean(), (1 / r).mean()))

def only(keys):
    return lambda c: sum(matching.WEIGHTS[k] * c[k] for k in keys if c[k] is

```

```

not None)

report('text only', only(['text']))
report('geo and time only', only(['geo', 'time']))
report('text + geo + time', only(['text', 'geo', 'time']))
report('text + geo + time + color', only(['text', 'geo', 'time', 'color']))
report('weighted score (all)', matching.weighted_score)
path = '/code/app/data/ranker.txt'
if os.path.exists(path):
    report('learned ranker', matching.load_ranker(path))

```

Run it with `docker compose exec backend python scripts/evaluate.py`. The synthetic data has no real photos, so the image and OCR rows only matter once you attach images (see the dataset README). Report the ablation table either way, and re-run it after you add images.

11.4 scripts/train_ranker.py (stretch goal)

This trains a LightGBM LambdaRank model on `candidate_pairs.csv` (positives plus twin, nearby and random negatives) using the train split, and validates on the val split.

```

import lightgbm as lgb
import numpy as np
import pandas as pd
from features import DATA, get, pair_components, vector

P = pd.read_csv(DATA + 'candidate_pairs.csv')

def build(split):
    d = P[P.split == split].sort_values('lost_id')
    X, y, groups = [], [], []
    for _, grp in d.groupby('lost_id', sort=False):
        for r in grp.itertuples():
            X.append(vector(pair_components(get('lost', r.lost_id),
            get('found', r.found_id))))
            y.append(int(r.label))
            groups.append(len(grp))
    return np.array(X), np.array(y), groups

Xtr, ytr, gtr = build('train')
Xva, yva, gva = build('val')
model = lgb.LGBMRanker(objective='lambdarank', n_estimators=300,
learning_rate=0.05, num_leaves=15)

```

```

model.fit(Xtr, ytr, group=gtr, eval_set=[(Xva, yva)], eval_group=[gva],
eval_at=[1, 5])
model.booster_.save_model('/code/app/data/ranker.txt')
print('saved ranker; feature importance:', dict(zip(['text', 'image',
'color', 'brand', 'geo', 'time', 'ocr'], model.feature_importances_)))

```

Then run `evaluate.py` again and compare the learned ranker with the weighted score on the test split. Keep the winner, and report both either way. A negative result that you explain is still a good result.

11.5 scripts/eval_verification.py (verification quality)

Replay the 1,052 synthetic claims through the real verifier and compare with the dataset labels.

```

import json

import pandas as pd
from features import DATA

from app.services.verification import decision_from, is_correct

claims = pd.read_csv(DATA + 'verification_claims.csv')
found = pd.read_csv(DATA + 'found_reports.csv').set_index('report_id')
items = pd.read_csv(DATA + 'items_ground_truth.csv').set_index('item_id')
name = {'approved': 'approve', 'manual_review': 'manual_review', 'rejected':
'reject'}
rows = []
for c in claims.itertuples():
    f = found.loc[c.found_id]
    cat = items.loc[f['item_id'], 'category']
    n = 0
    for i, (key, truth) in
enumerate(json.loads(f['hidden_details_json']).items(), 1):
        ans = getattr(c, 'a%d' % i)
        n += is_correct(cat, key, truth, ans if pd.notna(ans) else '')
    rows.append({'claim_type': c.claim_type, 'label': c.decision_label,
'pred': name[decision_from(n, 3)]})
df = pd.DataFrame(rows)
print(pd.crosstab(df.label, df.pred))
imp = df[df.claim_type == 'impostor']
print('false approval rate (impostors approved): %.3f' % (imp.pred ==
'approve').mean())
print('genuine full claims approved: %.3f' % (df[df.claim_type ==
'genuine_full'].pred == 'approve').mean())

```

The false approval rate is the headline number. Some impostors will approve by lucky guesses, because some question options have only four to six values. Report that honestly, and discuss how attempt limits, a larger question pool and the manual-review band reduce the risk.

12. Testing

backend/tests/test_matching.py

```
import datetime as dt
from types import SimpleNamespace as NS

from app.services.matching import pair_components, weighted_score

def mk(lat, lon, t, emb=(1.0, 0.0)):
    return NS(place=NS(latitude=lat, longitude=lon), event_time=t,
time_window_hours=1.0, txt_emb=list(emb),
img_emb=None, clip_txt_emb=None, color='black', brand='',
ocr_text='')

def test_near_and_recent_beats_far_and_old():
    t0 = dt.datetime(2026, 9, 1, 14, 0)
    lost = mk(12.9, 77.5, t0)
    near = mk(12.9001, 77.5001, t0 + dt.timedelta(minutes=30))
    far = mk(12.95, 77.55, t0 + dt.timedelta(days=4))
    assert weighted_score(pair_components(lost, near)) >
weighted_score(pair_components(lost, far))

def test_found_long_before_lost_gets_zero_time_score():
    t0 = dt.datetime(2026, 9, 1, 14, 0)
    assert pair_components(mk(12.9, 77.5, t0), mk(12.9, 77.5, t0 -
dt.timedelta(hours=5)))['time'] == 0.0
```

backend/tests/test_verification.py

```

from app.services.verification import decision_from, is_correct

def test_exact_and_embedded_answers_are_correct():
    assert is_correct('backpack', 'inner_lining_color', 'red', 'Red')
    assert is_correct('backpack', 'inner_lining_color', 'red', 'I think the
lining is red')
    assert not is_correct('backpack', 'inner_lining_color', 'red', '')

def test_decision_bands():
    assert decision_from(3, 3) == 'approved'
    assert decision_from(2, 3) == 'manual_review'
    assert decision_from(1, 3) == 'rejected'
    assert decision_from(0, 3) == 'rejected'

```

backend/tests/test_api_flow.py (end to end: found, lost, auto-match, claim, approve, handover)

```

import datetime as dt
import json
import uuid

from fastapi.testclient import TestClient
from sqlalchemy import select

from app.db import SessionLocal
from app.main import app
from app.models import Place

def signup(client, name):
    email = uuid.uuid4().hex[:8] + '@test.dev'
    r = client.post('/auth/register', json={'email': email, 'name': name,
'password': 'password123'})
    assert r.status_code == 200
    return {'Authorization': 'Bearer ' + r.json()['access_token']}

def test_full_recovery_flow():
    hidden = {'keychain': 'a red heart keychain', 'inner_lining_color':
'red', 'tag_or_sticker': 'no sticker'}
    with TestClient(app) as client:
        db = SessionLocal()
        place = db.scalar(select(Place).limit(1))
        if place is None:

```



```

        place = Place(name='Test Library', latitude=12.9, longitude=77.5)
        db.add(place)
        db.commit()
        owner, finder = signup(client, 'Owner'), signup(client, 'Finder')
        when = dt.datetime.utcnow() - dt.timedelta(hours=3)
        base = {'category': 'backpack', 'color': 'black', 'place_id':
place.id}
        found = client.post('/reports', headers=finder, data=**base, 'type':
'found',
            'description': 'Found a black Nike backpack near the library
entrance',
            'event_time': (when + dt.timedelta(hours=1)).isoformat(),
            'hidden_details': json.dumps(hidden))
        assert found.status_code == 200
        lost = client.post('/reports', headers=owner, data=**base, 'type':
'lost',
            'description': 'Lost my black Nike backpack with a white logo',
            'event_time': when.isoformat())
        matches = client.get('/reports/%d/matches' % lost.json()['id'],
headers=owner).json()
        mid = next(m['match_id'] for m in matches if m['other']['id'] ==
found.json()['id'])
        claim = client.post('/matches/%d/claim' % mid, headers=owner).json()
        res = client.post('/claims/%d/answers' % claim['claim_id'],
headers=owner, json={'answers': hidden}).json()
        assert res['result'] == 'approved' and res['handover_code']
        done = client.post('/claims/%d/confirm' % claim['claim_id'],
headers=finder, json={'code': res['handover_code']})
        assert done.status_code == 200

```

Run everything with `docker compose exec backend pytest -q`. Add tests for the negative paths too: a stranger cannot read hidden details, a wrong code is rejected, a third verification attempt is blocked, and an upload that is not an image is refused.

13. Security and privacy checklist

- Passwords hashed with argon2. Minimum length enforced. Tokens expire (default 2 hours).
- Add rate limiting with slowapi to login, register, search and claim routes (for example 5 per minute on login).
- Hidden details and claim answers are encrypted with Fernet. Keep the key outside the repository and back it up separately, because losing it makes stored details unreadable.

- Claimants never see hidden details, answer options or per-question correctness. Attempts are limited to 2 per match.
- Uploads: type, size and decode checks; re-encoding strips EXIF and GPS data; faces are blurred in the public copy; private originals are never served.
- CORS limited to your real frontend origin. HTTPS everywhere in production.
- Every sensitive action is audit logged: report creation, queue views, decisions, wrong handover codes.
- Contact happens only through the platform. No phone numbers or emails are displayed to other users.
- Moderator routes check the role on the server, not only in the UI.
- Data retention: delete closed reports and images after a fixed period (for example 90 days) and say so in a privacy notice.
- Abuse handling: the abuse endpoint plus a moderator view of flagged reports.

14. Deployment

Use a VPS with at least 4 GB of RAM, because the embedding models need memory. Very small free tiers often run out of memory when loading sentence-transformers.

1. Build the frontend

```
cd frontend
VITE_API_URL=/api npm run build
```

2. Caddyfile (replace the domain; Caddy gets HTTPS certificates automatically)

```
lostfound.example.com {
    encode gzip
    handle /api/* {
        uri strip_prefix /api
        reverse_proxy backend:8000
    }
    handle {
        root * /srv
        try_files {path} /index.html
        file_server
    }
}
```

3. docker-compose.prod.yml

```

services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: lf
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: lostfound
    volumes:
      - dbdata:/var/lib/postgresql/data
  backend:
    build: ./backend
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1
    env_file: .env
    volumes:
      - uploads:/data/uploads
    depends_on:
      - db
  caddy:
    image: caddy:2
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - ./frontend/dist:/srv
      - caddy_data:/data
    depends_on:
      - backend
volumes:
  dbdata:
  uploads:
  caddy_data:

```

4. Production .env: strong `DB_PASSWORD`, a new `JWT_SECRET` and `FERNET_KEY`, `DATABASE_URL` using the new password, `CORS_ORIGINS=https://lostfound.example.com`. Do not reuse the demo users. Create your own moderator account.

5. Launch

```
docker compose -f docker-compose.prod.yml up -d --build
```

6. Operate: nightly `docker compose exec db pg_dump -U lf lostfound > backup.sql` plus a copy of the uploads volume; firewall all ports except 22, 80 and 443; keep the OS and images updated; watch disk usage because images accumulate.

15. Build order and schedule

Week	Goal	Doc sections
1	Repo, Docker Compose, database, config, models	3, 4, 5
2	Auth, places, report create and list	8.1 to 8.3, 8.6, 8.8
3	Image upload, face blur, OCR	7.2
4	Embeddings, pgvector search, search page	7.1, 8.4, 10.5
5	Matching engine v1, match list UI, notifications	7.4, 10.8
6	Hidden details, verification, handover code	6, 7.5, 8.5, 10.9
7	UI polish, mobile layout, error states	10
8	Moderator queue and analytics	8.7, 10.10
9	Evaluation, ablations, learned ranker	11
10	Tests, security hardening, rate limits	12, 13
11	Deployment and a real pilot with 15 to 20 users	14
12	Report, final numbers, demo video	16

If you fall behind, cut stretch features first (learned ranker, extra analytics). Protect the verification flow, because it is your differentiator.

16. Demo script and report checklist

Five-minute demo

1. Log in as a finder and report a found backpack with a photo and three private answers.
2. Log in as the owner and report the lost bag using a vague description.
3. Show the suggested match with its confidence badge and per-signal breakdown.
4. Start verification: answer correctly (approved, code shown), then show an impostor attempt (rejected).
5. Confirm the handover as the finder with the code and show both reports closing.
6. Open the moderator view: a manual-review claim, then the analytics dashboard.

What to include in the written report

- Problem statement and the gap: multimodal matching plus challenge-based ownership verification.

- Architecture diagram and database schema.
- Dataset description: the synthetic generator, its noise model and its limits, plus your small real sample.
- Results: the ablation table (section 11.3), the weighted score versus the learned ranker, and the verification confusion matrix with the false approval rate.
- Latency: time from report submission to match suggestions.
- Threats to validity: synthetic text, no real photos in the benchmark, guessable question options, the negation weakness in answer checking.
- Ethics and privacy: what is stored, encrypted, blurred and deleted.

Likely questions from reviewers, and short answers

- Why not auto-declare a match? Because wrong releases harm people, so the system suggests and humans verify.
- How do you stop false claims? Private details the public never sees, limited attempts, a manual-review band and audit logs.
- Why combine CLIP and a text model? CLIP aligns photos with words, and the multilingual text model handles detailed and Hinglish descriptions better.
- What would you do with more time? Fine-grained attribute matching (logos, stickers), ID-card blurring, and a larger real dataset.